

Length and Algorithmic Generalization

English seminar study notes

These notes are an English companion to the seminar handout. The emphasis is on mathematical derivations that can be followed line by line, on separating theorem statements from interpretation, and on identifying the assumptions under which each claim is valid.

Primary readings

Kazemnejad et al., work on positional encoding and length generalization.

Zhou et al., work on RASP-L.

Huang et al., work on Limit Transformers.

Izzo et al., quantitative length-generalization bounds.

Hou et al., **Universal Length Generalization with Turing Programs**.

Contents

1	Problem setup and notation	3
1.1	What length generalization asks	3
1.2	Three questions to separate	3
2	Positional encoding and extrapolation	4
2.1	Why position enters first	4
2.2	Kazemnejad et al.: NoPE is not position-blind	4
2.3	PE extrapolation is not algorithm extrapolation	4
3	Zhou et al.: RASP-L and the simple-program hypothesis	5
3.1	RASP as a Transformer-like programming language	5
3.2	Why RASP-L restricts the language	5
3.3	The RASP-Generalization conjecture	5
3.4	Why scratchpads sometimes help	5
4	Huang et al.: turning identifiability into a theorem	6
4.1	Limit Transformers	6
4.2	PERIODIC and LOCAL structure	6
4.3	Idealized inference with a bounded regularizer	6
4.4	Eventual length generalization	6
5	Izzo et al.: how long is “long enough”?	8
5.1	Finite precision can turn softmax into hardmax	8
5.2	A quantitative one-layer bound	8
5.3	Why empirical frequencies are enough in the hardmax regime	9
5.4	Infinite precision	9
6	Algorithmic generalization as repeated dynamics	10
6.1	The same transition rule for longer time	10
6.2	Accumulation of local error	10
6.3	Fixed depth versus autoregressive depth	10
7	Turing Programs: a universal local format	11
7.1	Why task-specific scratchpads are unsatisfactory	11
7.2	Turing Programs	11
7.3	What universality does and does not mean	11
8	A unified view of length generalization	12
8.1	Four conditions	12
8.2	How the papers fit	12
8.3	Apparent contradictions	12
9	Presentation checklist	13
9.1	Equations worth keeping	13
9.2	Common mistakes	13

Bibliography**14**

1

Problem setup and notation

1.1 What length generalization asks

Suppose training sequences have length at most N_{train} , but evaluation uses much larger lengths. Length generalization asks whether the learned rule is *uniform in sequence length*: does the same computation continue to solve the task when new positions and longer dependency chains appear?

This is not ordinary i.i.d. generalization. The input dimension, number of positions, and sometimes the number of algorithmic steps all change with length.

1.2 Three questions to separate

1. **Representability:** does one parameterized model define the correct function for every length?
2. **Identifiability:** do bounded-length examples uniquely determine the intended uniform rule among plausible alternatives?
3. **Learnability/stability:** does training actually select that rule, and does repeated execution remain numerically stable?

How to say it in the presentation Length generalization is fundamentally an algorithm-selection problem. Fitting every sequence up to length N does not identify what the model should do at length $N + 1$ unless the hypothesis class contains a bias toward a uniform algorithm.

2

Positional encoding and extrapolation

2.1 Why position enters first

If an algorithm depends on relative order or distance, the model needs a way to distinguish positions. But having a positional feature defined at long indices is only a prerequisite; it does not prove that the same computation extrapolates.

2.2 Kazemnejad et al.: NoPE is not position-blind

A causal decoder without explicit positional embeddings still observes a position-dependent causal mask. The number of visible predecessors changes with index, and attention can exploit this asymmetry to construct absolute or relative position signals over a bounded range.

A schematic construction uses a uniform attention average over the visible prefix. If a designated feature has a single nonzero marker, the average can create a scalar proportional to $1/t$ at position t ; a feed-forward map can then transform this monotone quantity into an index-like code over a finite range. Combining such signals across positions yields relative-position information.

Caution The theoretical construction says that NoPE Transformers can represent positional information; it does not say NoPE is universally the best choice for extrapolation. Kazemnejad et al.’s empirical comparisons are task- and architecture-dependent.

2.3 PE extrapolation is not algorithm extrapolation

Even if RoPE, ALiBi, or another encoding is mathematically defined for arbitrary t , the learned attention/MLP computation can still depend on frequency ranges or index patterns only seen during training. Therefore

position code defined at long length $\nrightarrow$ correct long-length algorithm.

How to say it in the presentation The lesson from NoPE is not “remove positional encoding.” It is that positional information and length generalization are separate issues: even a causal mask can encode position, while a perfectly extrapolatable position code does not guarantee algorithmic extrapolation.

3

Zhou et al.: RASP-L and the simple-program hypothesis

3.1 RASP as a Transformer-like programming language

RASP describes sequence algorithms using primitives resembling attention selection, aggregation, and token-wise maps. Because program lines can often be compiled into a small number of Transformer layers, RASP program length is a Transformer-native notion of algorithmic complexity.

3.2 Why RASP-L restricts the language

Full RASP can express operations that are representable but difficult for standard training to discover robustly. RASP-L is a restricted fragment intended to better match operations that appear learnable by Transformers in synthetic algorithmic tasks.

3.3 The RASP-Generalization conjecture

The phenomenological hypothesis can be summarized by three ingredients:

1. **Realizability:** there exists one RASP-L program correct for all lengths;
2. **Simplicity:** that program is short/simple in the RASP-L language;
3. **Diversity:** the training distribution rules out even simpler shortcuts that fit short lengths but fail out of distribution.

When these hold, Transformers often show strong length generalization in the experiments.

3.4 Why scratchpads sometimes help

A scratchpad is useful when it rewrites a hard end-to-end mapping into a simpler local next-step rule. If each emitted token represents one local algorithmic transition, the RASP-L description of the next-token computation may become shorter. A verbose scratchpad that requires extra global counting or index arithmetic can instead make the program more complicated.

Caution RASP-Generalization is a conjectural explanatory framework, not a theorem that SGD minimizes RASP-L program length.

How to say it in the presentation Zhou et al. shift the question from parameter interpolation to program selection: does there exist a short Transformer-native program that works at every length, and does the training set eliminate shorter spurious shortcuts?

4

Huang et al.: turning identifiability into a theorem

4.1 Limit Transformers

To reason about all lengths with one object, Huang et al. define a limit-style Transformer class whose positional behavior is constrained so that infinitely many positions do not introduce infinitely many unrelated degrees of freedom.

4.2 PERIODIC and LOCAL structure

A positional mechanism is **PERIODIC** if, after some convention-dependent setup, its behavior repeats with finite period Δ . It is **LOCAL** if position-specific interactions depend only on a bounded relative window of radius τ .

The intuition is simple:

- **PERIODIC** compresses infinitely many absolute positions into finitely many position classes;
- **LOCAL** prevents independent long-range positional parameters from appearing at every new distance.

4.3 Idealized inference with a bounded regularizer

Consider a class $\mathcal{U}_n$ of candidate models that fit lengths up to n . An idealized procedure selects a candidate balancing empirical agreement and a bounded complexity regularizer. The exact paper formalism is more technical, but the role of the regularizer is to keep the candidate algorithm family compact enough that incorrect rules cannot hide forever.

4.4 Eventual length generalization

Under the paper's **PERIODIC/LOCAL** assumptions and a bounded-complexity target represented by one Limit Transformer, there exists a finite training length N_0 such that the idealized selection procedure identifies a predictor that generalizes to all longer lengths.

The proof intuition is an *identification in the limit* argument. Every incorrect bounded-complexity candidate differs from the true algorithm on some finite witness sequence. Because the effective algorithm class can be finitely compressed by periodicity, locality, and the regularizer, once training covers enough finite witnesses all incorrect candidates are eliminated.

Caution This is not a theorem that ordinary SGD on arbitrary Transformers always length-generalizes after seeing long enough sequences. The guarantee is for the structured

Limit-Transformer class and an idealized inference/selection procedure.

How to say it in the presentation Huang et al. formalize the identifiability intuition: constrain positional behavior and model complexity so that the space of uniform algorithms is controlled; then every wrong bounded-complexity rule is eventually exposed by a finite witness.

5

Izzo et al.: how long is “long enough”?

5.1 Finite precision can turn softmax into hardmax

Assume a one-layer Limit Transformer is computed with p bits of precision. After subtracting the maximum logit for numerical stability, suppose exponentiated terms below 2^{-p} round to zero. Let $\gamma(f) > 0$ be the minimum positive gap between a maximal and a non-maximal normalized logit.

With the length-dependent logit scaling in the model, a strictly submaximal term is bounded by

$$\exp(\log N (a - a_*)) \leq N^{-\gamma(f)}.$$

If

$$N \geq 2^{p/\gamma(f)},$$

then

$$N^{-\gamma(f)} \leq 2^{-p},$$

so all strictly submaximal contributions round to zero and the softmax behaves like a uniform hardmax over the maximizing set.

5.2 A quantitative one-layer bound

For one-layer, τ -local, Δ -periodic, translation-invariant models f, g evaluated at p -bit precision, let L denote the relevant norm scale and

$$\gamma = \min\{\gamma(f), \gamma(g)\}.$$

A representative theorem gives a sufficient training-length scale

$$N = O\left(\max\left\{2^{p/\gamma}, \frac{L^2 \Delta^7 |\Sigma|^6 \tau^2}{\varepsilon^2}\right\}\right)$$

such that agreement within ε on all inputs of length at most N implies $O(\varepsilon)$ agreement at arbitrary lengths, under the theorem assumptions.

The dependence is more important than the exact exponents: larger norm, larger period, larger alphabet, larger locality radius, or smaller target error requires more training length.

From the original paper The term $2^{p/\gamma}$ is a sufficient threshold arising in the proof’s finite-precision hardmax regime. It should not be interpreted as the exact empirical minimum training length.

5.3 Why empirical frequencies are enough in the hardmax regime

Once attention selects from finitely many token/position classes, a long sequence can be summarized by class frequencies

$$\pi_c(x) = \frac{\#\{j : \text{position } j \text{ is of class } c\}}{|x|}.$$

If a shorter sequence z satisfies

$$|\pi_c(z) - \pi_c(x)| \leq \delta$$

for every relevant class, attention averages remain close. With output Lipschitz scale L ,

$$\|f(x) - f(z)\| \lesssim L\delta.$$

The quantitative theorem constructs a short simulator z that preserves all relevant frequencies simultaneously.

5.4 Infinite precision

Without rounding, tiny attention weights do not disappear. The paper therefore analyzes position scalings that keep local suffix information visible at long length. For suitable two-layer classes, the sufficient length takes a polynomial form such as

$$N \lesssim (\max\{C(f), C(g)\} \varepsilon^{-1})^{\max\{\gamma(f)^{-1}, \gamma(g)^{-1}, 3\}},$$

where $C(f)$ is a paper-specific complexity measure incorporating norm/locality/alphabet factors.

How to say it in the presentation Huang proves that a finite identification length exists; Izzo asks how that length depends on precision, margins, norms, periodicity, locality, alphabet size, and accuracy. The proof strategy is to simulate a long forward pass by a shorter sequence that preserves the relevant sufficient statistics.

6

Algorithmic generalization as repeated dynamics

6.1 The same transition rule for longer time

Many algorithmic tasks can be written as a state recurrence

$$s_{t+1} = F(s_t, u_t).$$

Length extrapolation then means applying the same local transition F for more steps, rather than learning a new map for every possible length.

6.2 Accumulation of local error

Suppose an approximate transition satisfies

$$\|\hat{F}(s, u) - F(s, u)\| \leq \varepsilon$$

and F is L_F -Lipschitz in state. Let $e_t = \|\hat{s}_t - s_t\|$. Then

$$e_{t+1} \leq L_F e_t + \varepsilon.$$

Iterating from $e_0 = 0$ gives

$$e_T \leq \varepsilon \sum_{j=0}^{T-1} L_F^j.$$

If $L_F < 1$, errors remain bounded; if $L_F = 1$, they can grow linearly; if $L_F > 1$, they can grow geometrically. Thus uniform representability is not enough—execution stability matters.

6.3 Fixed depth versus autoregressive depth

A fixed-depth Transformer asked to solve a longer input in one shot must compress more algorithmic work into the same parallel depth. A scratchpad or autoregressive solver can instead spend more decoding steps as input length grows. Length generalization and computational-depth generalization are therefore closely linked.

How to say it in the presentation A model can learn the correct local rule and still fail at long lengths if local approximation errors amplify. Stable execution is the fourth ingredient after realizability, simplicity, and identifiability.

7

Turing Programs: a universal local format

7.1 Why task-specific scratchpads are unsatisfactory

RASP-L analyses often rely on finding a representation tailored to each algorithm. A natural question is whether arbitrary algorithms can be converted into a common local transition format that is friendly to length extrapolation.

7.2 Turing Programs

Hou et al. encode computation in a Turing-machine-like textual trace: each step records a local machine configuration and the next-token rule implements a bounded local transition. Because arbitrary Turing machines can be simulated by a RASP-style construction, the representation is universal at the level of computability.

7.3 What universality does and does not mean

The representability statement is

$$\begin{aligned} \text{arbitrary computable algorithm} &\longrightarrow \text{local Turing-program trace} \\ &\longrightarrow \text{Transformer/RASP simulation.} \end{aligned}$$

It does not prove that finite-data SGD universally learns the correct simulation or that the resulting trace is always efficient.

How to say it in the presentation The Turing-Programs idea is to stop inventing a different scratchpad for every task and instead translate algorithms into a common local transition language. This makes the length-generalization bias explicit, while leaving learnability as a separate question.

8

A unified view of length generalization

8.1 Four conditions

A useful synthesis is:

1. **Uniform realizability:** one rule is valid for all lengths.
2. **Bounded/simple representation:** the rule has controlled complexity in a Transformer-native language.
3. **Identifiability/diversity:** short-length data eliminates spurious shortcuts.
4. **Stable execution:** approximation errors do not explode when the rule is repeated longer.

8.2 How the papers fit

Paper line	Main question	Contribution
Kazemnejad et al.	does positional encoding control extrapolation?	empirical comparison + NoPE position construction
Zhou et al.	which learned rules extrapolate?	short RASP-L program conjecture
Huang et al.	can identifiability be formalized?	PERIODIC/LOCAL + bounded-complexity eventual guarantee
Izzo et al.	how large must training length be?	quantitative sufficient bounds
Hou et al.	can one use a universal algorithmic format?	Turing Programs and RASP simulation

8.3 Apparent contradictions

“NoPE can encode position” and “positional restrictions are needed” are compatible: the first is a finite-range representability statement, while the second controls uniform behavior over unbounded lengths.

“A task has a short RASP-L representation” and “the task may fall outside one Limit-Transformer subclass” can also both be true because the formalisms impose different primitives and positional restrictions.

How to say it in the presentation The common theme is not a particular positional encoding. It is selecting a compact, uniform, identifiable, and stable algorithm whose definition does not change when new positions appear.

9

Presentation checklist

9.1 Equations worth keeping

fit up to $N \not\approx$ unique behavior beyond N ,

$$N^{-\gamma} \leq 2^{-p} \quad \text{when } N \geq 2^{p/\gamma},$$

$$N = O\left(\max\left\{2^{p/\gamma}, \frac{L^2 \Delta^7 |\Sigma|^6 \tau^2}{\varepsilon^2}\right\}\right),$$

$$\pi_c(x) = \frac{\#\{j : j \in c\}}{|x|},$$

$$e_{t+1} \leq L_F e_t + \varepsilon,$$

$$e_T \leq \varepsilon \sum_{j=0}^{T-1} L_F^j.$$

9.2 Common mistakes

- Calling NoPE universally superior based on a finite set of experiments.
- Presenting the RASP-L conjecture as a proved SGD theorem.
- Reading Huang's eventual result as a guarantee for arbitrary Transformers and ordinary training.
- Treating Izzo's sufficient training-length bound as a necessary threshold.
- Confusing Turing-program representability with universal finite-data learnability.

How to say it in the presentation Thirty-second conclusion: length generalization succeeds when the model represents one uniform algorithm, the hypothesis bias makes that algorithm simple, training data identifies it against short-length shortcuts, and its execution remains stable as the number of positions or algorithmic steps grows.

Bibliography

- Kazemnejad, A. et al. *The Impact of Positional Encoding on Length Generalization in Transformers*. NeurIPS, 2023.
- Zhou, H. et al. Work on RASP-L and length generalization, ICLR 2024.
- Huang, Y. et al. Work on Limit Transformers and provable length generalization, ICLR 2025.
- Izzo, Z. et al. Work on quantitative training-length bounds for length generalization, ICLR 2026.
- Hou, B. et al. *Universal Length Generalization with Turing Programs*. ICML, 2025.